

Design Patterns Architecture Guide

Project: Sarasavi Pages – Web-Based Bookstore Management System

Institution: SLIIT – Faculty of Computing (2nd Year, 1st Semester)

Lectures Grounded: Week 11 Design Patterns Part I & Part II

1. Executive Summary: Viva Quick Reference Table

Below is the summary mapping each pattern taught in SE2030 lecture slides to its actual implementation in Sarasavi Pages:

Pattern	Type	Lecture Source	Sarasavi Pages Usage	Source File
1. Singleton	Creational	Part I (Slides 14–27)	Spring IoC Singletons, JwtUtil token crypto, background sync scheduler	JwtUtil.java, SyncScheduler.java
2. Observer	Behavioral	Part I (Slides 28–42)	Security audit logging, MongoDB read-replica sync, inventory stock threshold alerts	AuditLogService.java, InventoryServiceImpl.java
3. Strategy	Behavioral	Part II (Slides 2–24)	Multi-channel payment (Card, Bank Slip, Cash on Delivery), coupon discount calculations	PaymentServiceImpl.java, PromotionServiceImpl.java
4. Factory Method	Creational	Part II (Slides 25–36)	Static Factory Methods in DTOs (omits sensitive passwords), MongoDB-to-DTO factory	StaffResponse.java, PaymentMapper.java
5. Decorator	Structural	Part II (Slides 37–49)	Spring Security Filter Chain dynamically decorating HTTP requests with JWT principal	JwtAuthFilter.java, SecurityConfig.java
Bonus: Builder	Creational	GoF Standard	Fluent, immutable entity instantiation across all models via Lombok	Staff.java, Payment.java

Bonus: State	Behavioral	GoF Standard	Single Finite State	Behavioral Strategy/Template
			Machine: PENDING → PAID → REFUNDED status transitions	

2. Pattern 1: Creational – Singleton Pattern

Lecture Reference: Week11_Lecture_DesignPattern-Part01.pdf (Slides 14–27)

Theory: Ensures a class has only one instance and provides a global point of access. It encapsulates the instantiation mechanism and avoids redundant resource consumption (e.g. database connections, socket listeners, cryptographic managers).

How it is used in Sarasavi Pages:

- **Spring Framework IoC:** By default, Spring Boot registers and maintains all `@Service`, `@Repository`, and `@Component` beans as singletons.
- **Cryptographic Utility (`JwtUtil.java`):** A single shared HMAC signing key instance is kept in memory. All HTTP requests needing JWT validation or generation leverage this single instance rather than generating new keys for each client request.
- **Background Workers (`SyncScheduler.java` & `BackupScheduler.java`):** Single execution instances handle recurring tasks every 5 minutes.

```
// Centralized, singleton cryptographic helper in Spring
@Component
public class JwtUtil {
    private final Key key; // Single in-memory key instance

    public String generateToken(String username, String role) { ... }
    public boolean validateToken(String token) { ... }
}
```

3. Pattern 2: Behavioral – Observer Pattern

Lecture Reference: Week11_Lecture_DesignPattern-Part01.pdf (Slides 28–42)

Theory: Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically without tight coupling.

How it is used in Sarasavi Pages:

- **Security & Administrative Audit Logging:** In Module M1, when staff actions occur (creating accounts, updating permissions, logging in), `AuditLogService` acts as an observer that captures and saves the event to the `audit_log` table without blocking the core business action.
- **Dual-Tier Database Synchronization:** The `SyncScheduler` observes changes in the primary MSSQL tables and pushes mirrors to MongoDB read-replicas for disaster recovery.
- **Stock Adjustment Alerts:** In Module M4, inventory stock deductions notify observers to record adjustments in `StockAdjustmentLog` and transition stock status tags (`IN_STOCK`, `LOW_STOCK`, `OUT_OF_STOCK`).

```
// Subject notifies the audit observer upon staff creation
auditLogService.log(
    performedBy,
    AuditAction.STAFF_CREATED,
```

```
saved.getUsername(),
"Staff created: " + saved.getFullName() + " (" + saved.getRole() + ")"
);
```

4. Pattern 3: Behavioral – Strategy Pattern

Lecture Reference: DesignPattern PartII.pdf (Slides 2–24)

Direct Lecture Match: Slide 10 of Part II explicitly features "**Example 01: E-commerce Payment**" with interchangeable payment methods as the primary example!

Theory: Defines a family of algorithms, encapsulates each one, and makes them interchangeable at runtime. The algorithm varies independently from clients that use it, eliminating long `if-else` chains.

How it is used in Sarasavi Pages:

- **Multi-Channel Payment Processing (Module M2):** Customers choose between `CARD` (Credit/Debit Card gateway simulation), `BANK_TRANSFER` (Bank deposit slip verification), `DIGITAL_WALLET`, and `CASH_ON_DELIVERY`. Each method follows a distinct processing and status update strategy.
- **Discount Calculation Strategy (Module M6):** In `PromotionServiceImpl`, coupons evaluate discounts using either a percentage algorithm or a capped maximum ceiling algorithm based on policy.

```
// Payment Strategy Handling in PaymentServiceImpl.java
public PaymentResponse recordPayment(PaymentCreateRequest request) {
    PaymentMethod method = request.getPaymentMethod();

    if (method == PaymentMethod.CARD) {
        boolean approved = simulateGatewayCharge(request.getAmount());
        payment.setStatus(approved ? PaymentStatus.PAID : PaymentStatus.FAILED);
    } else if (method == PaymentMethod.BANK_TRANSFER) {
        payment.setStatus(PaymentStatus.PENDING); // Awaiting slip verification
    } else if (method == PaymentMethod.CASH_ON_DELIVERY) {
        payment.setStatus(PaymentStatus.PENDING); // Paid upon physical delivery
    }
}
```

5. Pattern 4: Creational – Factory Method Pattern

Lecture Reference: DesignPattern PartII.pdf (Slides 25–36)

Theory: Defines an interface or method for creating an object, but encapsulates the creation details and hides instantiation complexity from callers.

How it is used in Sarasavi Pages:

- **Static Factory Methods in DTOs (StaffResponse.java):** Hides internal entity structure and sensitive fields (e.g. BCrypt password hashes are never exposed to the client). Controllers simply call factory methods `StaffResponse.from(staff)` or `StaffResponse.fromMirror(mirror)`.
- **Dedicated Entity Mappers:** `PaymentMapper.java` acts as an object factory transforming request DTOs into JPA Entities.

```
// Static Factory Methods in StaffResponse.java
public static StaffResponse from(Staff staff) {
    StaffResponse dto = new StaffResponse();
```

```

        dto.setId(staff.getId());
        dto.setUsername(staff.getUsername());
        dto.setFullName(staff.getFullName());
        dto.setEmail(staff.getEmail());
        dto.setEmployeeId(staff.getEmployeeId()); // Corporate EMP-ID
        dto.setRole(staff.getRole());
        dto.setActive(staff.isActive());
        // Notice: Password is intentionally excluded!
        return dto;
    }

```

6. Pattern 5: Structural – Decorator Pattern

Lecture Reference: DesignPattern PartII.pdf (Slides 37–49)

Theory: Attaches additional responsibilities and behavior to an object dynamically at runtime without modifying the underlying class structure.

How it is used in Sarasavi Pages:

- **Spring Security Filter Chain (JwtAuthFilter.java):** Implements the classic Decorator / Intercepting Filter pattern. Incoming HTTP servlet requests are dynamically intercepted, examined for Bearer tokens, and decorated with authenticated Security Context principals before delegating to target controllers.

```

// Decorator Filter in Spring Security
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
        FilterChain filterChain) {
        // Decorates request with authenticated principal & roles
        SecurityContextHolder.getContext().setAuthentication(authToken);
        // Proceeds along the wrapped filter chain
        filterChain.doFilter(request, response);
    }
}

```

7. Bonus Patterns Implemented in Sarasavi Pages

A. Builder Pattern (Creational)

Used across all entities and DTOs using Lombok @Builder. Provides clear, fluent, readable object construction without fragile 10-parameter constructors.

```

Staff staff = Staff.builder()
    .username(username)
    .password(passwordEncoder.encode(rawPassword))
    .fullName(request.getFullName().trim())
    .email(request.getEmail().trim())
    .employeeId(empId)
    .role(request.getRole())
    .active(true)
    .build();

```

B. State Pattern (Behavioral – Finite State Machine)

In PaymentServiceImpl.java, a strict transition matrix ensures payments only transition along legal business

```
fecycles (PENDING → PAID → REFUNDED):
```

```
private static final Map<PaymentStatus, Set<PaymentStatus>> ALLOWED_TRANSITIONS = new
EnumMap<>(PaymentStatus.class);
static {
    ALLOWED_TRANSITIONS.put (PaymentStatus.PENDING, EnumSet.of (PaymentStatus.PAID,
PaymentStatus.FAILED, PaymentStatus.VOIDED));
    ALLOWED_TRANSITIONS.put (PaymentStatus.PAID, EnumSet.of (PaymentStatus.REFUNDED));
    ALLOWED_TRANSITIONS.put (PaymentStatus.FAILED, EnumSet.of (PaymentStatus.PENDING,
PaymentStatus.VOIDED));
    ALLOWED_TRANSITIONS.put (PaymentStatus.REFUNDED, EnumSet.noneOf (PaymentStatus.class));
    ALLOWED_TRANSITIONS.put (PaymentStatus.VOIDED, EnumSet.noneOf (PaymentStatus.class));
}
```

8. Viva Q&A Cheat Sheet (Model Questions & Answers)

Q1: What design patterns have you used in your bookstore system?

English: "We implemented 5 key patterns covered in our SE2030 lectures: **Singleton** (Spring IoC & JwtUtil), **Observer** (Security Audit Logging & DB Replication), **Strategy** (Multi-channel Payments in Module 2 and Coupon Discounts in Module 6), **Factory Method** (Safe DTO construction in StaffResponse), and **Decorator** (Spring Security Filter Chain). We also utilized the **Builder** pattern via Lombok and the **State** pattern for payment transitions."

සිංහල සාරාංශය: "අපි SE2030 lectures වල නියෙන Singleton, Observer, Strategy, Factory Method, සහ Decorator patterns පහම අපේ bookstore එකේ ප්‍රායෝගිකව යොදාගෙන තියෙනවා."

Q2: Where is the Strategy Pattern implemented in your code?

English: "In Module M2 (Payment Management), we have interchangeable payment strategies: Card gateway simulation, Bank Slip transfer, and Cash on Delivery. Depending on customer checkout choice, the service selects and executes the appropriate verification algorithm without tightly coupling the checkout controller to gateway implementations."

File to open: `PaymentServiceImpl.java` and `PaymentMethod.java`

Q3: How does the Observer Pattern work in your system?

English: "We use it in Audit Logging. When a state change happens in StaffService (like staff creation or login), the service notifies the AuditLogService observer, which persists an immutable log to MSSQL. It is also used in our background SyncScheduler to observe and replicate changes from primary MSSQL to MongoDB."

File to open: `AuditLogService.java` and `StaffService.java` (line 344)

Q4: Why did you use Factory Methods for your DTOs?

English: "Our static factory method `StaffResponse.from(...)` hides the construction complexity, guarantees that sensitive information such as BCrypt password hashes are stripped before leaving the backend, and allows constructing identical DTOs polymorphically from both relational entities and MongoDB documents."

File to open: `StaffResponse.java` (lines 30–65)

Code File Index & Quick Links for the Presentation

Component	File Path (Relative)	Highlighted Feature
Payment Strategy	backend/.../m2_anaf_payment/service/impl/ PaymentServiceImpl.java	Runtime payment method strategy branching
Audit Observer	backend/.../m1_gunathilaka_adminstaff/service/ AuditLogService.java	Decoupled security activity observer
DTO Factory Method	backend/.../m1_gunathilaka_adminstaff/dto/ StaffResponse.java	Static factory <code>from()</code> and <code>fromMirror()</code>
Crypto Singleton	backend/.../config/JwtUtil.java	Shared key and cryptographic validation singleton
Security Decorator	backend/.../config/JwtAuthFilter.java	Request wrapping & security principal decoration
State Pattern Matrix	backend/.../m2_anaf_payment/service/impl/ PaymentServiceImpl.java	ALLOWED_TRANSITIONS Finite State Machine

--	--	--	--	--

--	--	--	--	--

--	--	--	--	--